# A Hardware Descriptions

This section provides an overview of the basic hardware components used in our benchmark, including their functions and pin configurations.

## LED

A Light Emitting Diode (LED) is a basic component used for visual indication

. **Pins:**

- `A` – Anode (positive terminal)
- `C` – Cathode (negative terminal)

## Pushbutton

A pushbutton is a momentary switch used for digital input.

**Pins:**

- `1.l`, `1.r` – First contact terminals
- `2.l`, `2.r` – Second contact terminals

## RGB LED

An RGB LED combines red, green, and blue LEDs in a single package to produce different colors.

**Pins:**

- `R`, `G`, `B` – Controls red, green, and blue channels respectively
- `COM` – Common cathode (connected to ground)

## Seven-Segment Display

A seven-segment display is used for numerical output.

**Pins:**

- `A–G` – Individual segment controls
- `DP` – Decimal point
- `COM.1`, `COM.2` – Common anode pins

## LED Bar Graph

An LED bar graph displays signal levels or statuses with multiple LEDs in a row.

**Pins:**

- `A1–A10` – Anodes of each LED
- `C1–C10` – Cathodes of each LED

## Slide Potentiometer

A slide potentiometer provides analog output based on position.

**Pins:**

- `GND` – Ground
- `SIG` – Analog output
- `VCC` – Supply voltage

**Attribute:** The output value ranges from 0 to 1023.

## Resistor

A resistor limits current flow in a circuit.

**Pins:**

- `1`, `2` – Connection terminals

**Attribute:** Resistance value: $1000\Omega(1k\Omega)$

## 74HC595 Shift Register

A shift register expands the number of digital outputs using serial data input.

**Pins:**

- `DS` – Serial data input
- `SHCP` – Shift register clock input
- `STCP` – Storage register clock input (latch)
- `OE` – Output enable (active low)
- `Q0–Q7` – Parallel outputs
- `MR` – Master reset (active low)
- `GND` – Ground
- `VCC` – Supply voltage (typically 5V)

## Servo Motor

A servo motor allows precise angular positioning.

**Pins:**

- `PWM` – Control signal
- `V+` – Power supply (5V)
- `GND` – Ground

# B Platform Descriptions

This section outlines the hardware platforms used in our benchmark, including general-purpose input/output (GPIO) pins and power configurations.

## Raspberry Pi Pico

A microcontroller board based on the RP2040 chip.

**Pins:**

- `GP0–GP22` – Digital GPIO pins
- `GP26–GP28` – Analog input pins
- `GND.1–GND.8` – Ground connections
- `VSYS`, `VBUS`, `3V3` – Power supply inputs and outputs

## Arduino Uno Rev3

A popular microcontroller board based on the ATmega328P.

**Pins:**

- `0–13` – Digital GPIO pins
- `A0–A5` – Analog input pins
- `GND.1–GND.3` – Ground pins
- `VIN` – External power input (7–12V)
- `5V` – 5V power output

## Arduino Mega

A microcontroller board with expanded I/O capabilities, based on the ATmega2560.

**Pins:**

- `0–53` – Digital GPIO pins
- `A0–A15` – Analog input pins
- `GND.1–GND.5` – Ground pins
- `VIN` – External power input (7–12V)
- `5V` – 5V power output

**Note:** Pins `2–13`, `44–46` support hardware PWM output (15 channels total).

### ESP32-DevKitC V4

A development board based on the ESP32 microcontroller with integrated Wi-Fi and Bluetooth.

**Pins:**

- `0, 2, 4, 5, 12–19, 21–23, 25–27, 32, 33, 34, 35` – GPIO pins
- `GND.1–GND.3` – Ground pins
- `3V3` – 3.3V power output

## C Data Annotation and Quality Control

### Human Annotation

In this section, we provide the detailed process for human annotation of our benchmark.

Initially, we recruited two bachelor's degree holders as annotators. After undergoing rigorous training, each annotator independently performed the annotations. To ensure quality control, an additional pair of bachelor's degree holders reviewed all annotated content. The guidelines for our annotation primarily cover the following aspects:

(1) **Optimal Hardware Resource Utilization Principle:** the problem description must ensure the complete and efficient utilization of all available hardware resources.

(2) **State Duration Stability Specification:** with the exception of instantaneous interaction scenarios (e.g., button-triggered interactions), all device states must maintain a stable duration of no less than two seconds.

(3) **The Test Validity Criteria:** test cases must satisfy completeness criteria, meaning they should comprehensively verify the functional characteristics and correctness of the target problem. Each test case should be designed with a specific validation objective in mind.

These guidelines aim to ensure the quality and effectiveness of the test case repository while enhancing the stability and reproducibility of the testing process.

### Quality Control

To ensure the correctness of the benchmark, each example was independently annotated by two annotators to minimize subjective bias. Upon completing their annotations, the two annotators cross-verified each other's results. Once both annotators reached a consensus, the annotated content was reviewed by two additional volunteers for further validation.

Volunteers meticulously examined the annotations and corrected errors. Through discussion, they selected the most accurate versions. The finalized annotations include the problem statements, corresponding solutions, and test cases. During the review phase, volunteers adhered to the following standardized quality assessment protocol:

(1) **Problem Description Evaluation:** Assess the completeness and clarity of the problem formulation.Problem Description Evaluation: The problem formulation must be complete and clear, with unambiguous requirements covering all specifications.

(2) **Test Case Validation:** Verify the logical soundness of the test case design, ensuring alignment with the problem's functional and edge-case requirements. Evaluate whether the test suite provides sufficient validation for algorithmic correctness and robustness.

(3) **Stability Testing:** Perform a minimum of five independent test executions under controlled conditions. We mitigated external interference (e.g., computational resource contention) to ensure reproducible results. Document response times and output consistency for all runs to assess system reliability.

## D Dataset Statistics

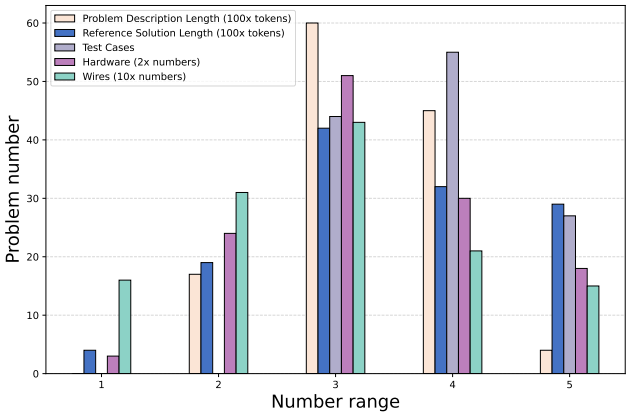



**Figure 6: The dataset statistics. The horizontal axis number i means [(i-1)*interval,i*interval), for example (100x tokens) means interval is 100.**

In this section, we present statistical insights into our dataset, covering Problem Description Length, Reference Solution Length, the number of electronic components, and the number of Wires used. As shown in Figure 6, most problem descriptions and reference solutions fall within the 200–400 tokens range. The distribution of test cases is relatively uniform, with a majority of problems containing four test cases. Additionally, the number of electronic components is mostly concentrated between 4 and 8, while the number of wires predominantly falls in the 10–40 range.

## E The Relationship Between LLM Capabilities and the Number of Components

In this section, we examine how the performance of LLMs varies with the number of component types required in a task. Specifically, we compute the pass@1 score of each model across four tasks, grouped by the number of component types involved. The detailed results are presented in Table 9.

Our analysis shows a clear trend: model performance generally decreases as the number of component types increases. The average pass@1 scores for tasks involving 1, 2, and 3 component types are 57.0, 38.7, and 35.2, respectively, indicating that tasks with a greater variety of component types pose more challenges for LLMs.

| Model | Task | 1 | 2 | 3 |
|---|---|---|---|---|
| Claude-3.7-Sonnet | with connection | 0.62 | 0.45 | 0.31 |
| | without connection | 0.68 | 0.46 | 0.33 |
| | translate Pi-Pico | 0.72 | 0.68 | 0.69 |
| | translate ESP32 | 0.58 | 0.39 | 0.67 |
| O3-Mini | with connection | 0.53 | 0.47 | 0.47 |
| | without connection | 0.52 | 0.43 | 0.13 |
| | translate Pi-Pico | 0.81 | 0.69 | 0.64 |
| | translate ESP32 | 0.71 | 0.24 | 0.67 |
| DeepSeek-Reasoner | with connection | 0.75 | 0.51 | 0.38 |
| | without connection | 0.63 | 0.53 | 0.29 |
| | translate Pi-Pico | 0.63 | 0.55 | 0.56 |
| | translate ESP32 | 0.33 | 0.29 | 0.13 |
| QwQ-32B | with connection | 0.53 | 0.41 | 0.27 |
| | without connection | 0.71 | 0.51 | 0.36 |
| | translate Pi-Pico | 0.44 | 0.54 | 0.44 |
| | translate ESP32 | 0.25 | 0.25 | 0.00 |
| DeepSeek-R1-Distill-Llama-70B | with connection | 0.40 | 0.34 | 0.50 |
| | without connection | 0.48 | 0.29 | 0.27 |
| | translate Pi-Pico | 0.27 | 0.04 | 0.10 |
| | translate ESP32 | 0.00 | 0.00 | 0.00 |
| DeepSeek-R1-Distill-Qwen-32B | with connection | 0.38 | 0.38 | 0.23 |
| | without connection | 0.33 | 0.26 | 0.30 |
| | translate Pi-Pico | 0.33 | 0.25 | 0.25 |
| | translate ESP32 | 0.67 | 0.33 | 0.50 |
| DeepSeek-Chat | with connection | 0.44 | 0.33 | 0.40 |
| | without connection | 0.39 | 0.24 | 0.13 |
| | translate Pi-Pico | 0.63 | 0.40 | 0.31 |
| | translate ESP32 | 0.00 | 0.00 | 0.00 |
| Llama-3.3-70B-Instruct | with connection | 0.38 | 0.31 | 0.33 |
| | without connection | 0.32 | 0.22 | 0.06 |
| | translate Pi-Pico | 0.32 | 0.22 | 0.06 |
| | translate ESP32 | 0.00 | 0.00 | 0.00 |
| Qwen2.5-32B-Instruct | with connection | 0.39 | 0.26 | 0.31 |
| | without connection | 0.41 | 0.24 | 0.21 |
| | translate Pi-Pico | 0.38 | 0.29 | 0.13 |
| | translate ESP32 | 0.00 | 0.00 | 0.00 |
| StarCoder2-15B | with connection | 0.13 | 0.24 | 0.11 |
| | without connection | 0.18 | 0.04 | 0.00 |
| | translate Pi-Pico | 0.19 | 0.00 | 0.00 |
| | translate ESP32 | 0.00 | 0.00 | 0.00 |

**Table 9: Performance of different LLMs on embedded system tasks with 1-3 numbers of component types.**

# F Why EmbedBench Is a Benchmark

In software engineering, a benchmark is more than an evaluation dataset; it is expected to support fair comparison, reproducibility, and cumulative research. Prior work has articulated explicit criteria for what qualifies a benchmark, most notably the desiderata proposed by Sim et al. [27]. In this section, we argue that EmbedBench qualifies as a benchmark by demonstrating that it satisfies these established criteria.

**Accessibility** EmbedBench is designed to be easy to obtain and straightforward to use. All benchmark tasks, test cases, reference implementations, and evaluation scripts will be publicly released with detailed documentation and Docker support. This packaging enables researchers and practitioners to readily apply the benchmark without requiring specialized hardware or proprietary tools.

**Affordability** In terms of affordability, the benchmark relies on a simulated and controllable embedded-system environment, which eliminates the need for physical hardware and significantly reduces experimental cost. The evaluation process is fully automated, minimizing human effort while ensuring consistent measurement across different LLMs. These design choices lower the barrier to adoption and make repeated benchmarking feasible.

**Clarity** The benchmark specification is clear, self-contained, and explicitly defined. Each task in EmbedBench includes a natural-language problem description, a precise hardware configuration, and well-defined correctness criteria. The evaluation protocol leaves little room for ambiguity or loopholes, reducing the risk of misinterpretation and increasing the credibility of reported results.

**Relevance** The benchmark tasks are constructed to reflect realistic embedded-system development scenarios. They cover multiple roles commonly encountered in practice, including programmer, system architect, and integrator. Rather than relying on synthetic or isolated code snippets, the tasks require models to reason about hardware components, pin assignments, and system-level constraints, closely mirroring real-world embedded development workflows. As a result, the benchmark evaluates capabilities that are directly relevant to practical deployment of LLMs in embedded systems.

**Solvability** A successful benchmark must be challenging yet solvable. Empirically, our evaluation of ten representative LLMs shows that even the strongest models achieve only around 50% pass@1 accuracy. This indicates that the tasks are non-trivial, while still being within reach of current state-of-the-art models. Such a difficulty level enables meaningful differentiation among techniques and avoids both degenerate success and universal failure.

**Portability** EmbedBench is specified at a sufficiently high level of abstraction to avoid bias toward a particular model family or implementation strategy. The tasks are model-agnostic and do not assume specific prompting methods, training data, or architectural features. Moreover, the benchmark can be readily extended to different embedded platforms by modifying the hardware configuration descriptions and validation scripts, supporting portability across tools, models, and system settings.

**Scalability** The benchmark is designed to scale with the maturity of LLM-based techniques. It can be applied to lightweight research prototypes as well as large commercial models. Furthermore, the task set and evaluation framework naturally support extension in

both size and complexity, enabling future work to incorporate more advanced hardware configurations, longer workflows, or multi-stage design tasks as the field progresses.

## G Thinking Process of 7-Segment Display

**Part of Thinking Process in 7-Segment Display**

...
For digit 1, the bits would be:
Bit7 (Q7): 0 Bit6 (Q6): 1 Bit5 (Q5): 1 Bit4 (Q4): 1 Bit3 (Q3): 1 Bit2 (Q2): 0 Bit1 (Q1): 0 Bit0 (Q0): 1
So binary 0 1 1 1 1 0 0 1 → which is 0b01111001 (0x79). Wait, but that would set Q0 to 1 (A off), Q1 to 0 (B on), Q2 to 0 (C on), and the rest (Q3-6 to 1, Q7 0). So segments B and C are on. That's correct. So the byte for digit 1 is 0x79.
**This process is tedious,** but necessary. Let's do this for each digit:
Digit 0: 0x40 (binary 01000000) → A-F on, G off.
Digit 1: 0x79 (0b01111001) → B and C on.
Digit 2: Segments A, B, G, E, D → A (Q0=0), B (Q1=0), G (Q6=0), E (Q4=0), D (Q3=0). So which bits are 0? Q0, Q1, Q3, Q4, Q6. So the byte bits:
Q7:0 Q6:0 (G on) Q5:1 (F off) Q4:0 (E on) Q3:0 (D on) Q2:1 (C off) Q1:0 (B on) Q0:0 (A on)
So bits: 0 0 1 0 0 1 0 0 → binary 00100100 → 0x24. Wait, let's write it as 8 bits: 0 0 1 0 0 1 0 0 → that's 0x24 (36). Wait:
Q7:0 → bit7:0 Q6:0 → bit6:0 Q5:1 → bit5:1 Q4:0 → bit4:0 Q3:0 → bit3:0 (Wait, D is on, so Q3 is 0 → bit3 is 0) Wait, no. Wait, for digit 2, D is on (Q3=0). So Q3 is 0 → bit3 is 0. Let me re-calculate:
Segments for digit 2: A, B, G, E, D → A (Q0=0), B (Q1=0), G (Q6=0), E (Q4=0), D (Q3=0). Other segments (C and F) are off → Q2=1, Q5=1.
So the bits:
Q7:0 Q6:0 Q5:1 Q4:0 Q3:0 Q2:1 Q1:0 Q0:0
Which is 0 0 1 0 0 1 0 0 → binary 00100100 → 0x24. So yes, 0x24 for digit 2.
Hmm, **this is time-consuming**. Maybe there's a better way to approach this. Alternatively, perhaps we can create a lookup table using a helper function that for each digit, defines which segments are on (LOW), and then build the byte accordingly.
Alternatively, perhaps using a standard 7-segment code table for common anode displays. For example, the standard byte codes for a common anode 7-segment display connected to a shift register.
Wait, the standard segment codes for common anode (assuming segments a to g are connected to Q0 to Q6):
For example, the standard code for 0 is 0x03 (binary 00000011) but that depends on the wiring. Wait, perhaps this approach is error-prone. Let me look for a standard 7-segment code table.
Upon checking, the standard byte patterns for a common anode 7-segment display (assuming segments are a, b, c, d, e, f, g, dp connected to Q0-Q7), but in our case, the segments are connected as:
Q0 → A Q1 → B Q2 → C Q3 → D Q4 → E Q5 → F Q6 → G
So each segment corresponds to a specific Q output. The standard codes for the digits would then need to be mapped to these Q outputs. Let's list the segments for each digit and then map to the corresponding bits.
...

## H Prompt and Example of R1-Retrival

**R1-Retrival Prompt**

{ prompt same as "w/ schematics" or "w/o schematics" }

### Retrieve Code
Here are some retrieved code snippets that may be helpful for solving the problem:

**LED**

```
// Define the pin where the LED is connected
const int ledPin = 2;

void setup() {
  // Initialize the LED pin as an output
  pinMode(ledPin, OUTPUT);
  // Ensure the LED is off initially
  digitalWrite(ledPin, LOW);
}

void loop() {
  // Turn the LED on for 1 second
  digitalWrite(ledPin, HIGH);
  delay(1000);

  // Turn the LED off for 2 seconds
  digitalWrite(ledPin, LOW);
  delay(2000);

  // Turn the LED on for 3 seconds
  digitalWrite(ledPin, HIGH);
  delay(3000);

  // Turn the LED off for 3 seconds
  digitalWrite(ledPin, LOW);
  delay(3000);

  // The sequence will repeat indefinitely due to the
  ↪ loop() function
}
```

**PushButton**

```
// Define the pins for the RGB LED and the push button
const int redPin = 2;    // Red pin of the RGB LED
const int greenPin = 3;  // Green pin of the RGB LED
const int bluePin = 4;   // Blue pin of the RGB LED
const int buttonPin = 5; // Push button pin

// Variables to manage the button state and debouncing
int buttonState = HIGH;      // Current state of the button
int lastButtonState = HIGH;  // Previous state of the
↪ button
unsigned long lastDebounceTime = 0; // Timestamp of the
↪ last button state change
const unsigned long debounceDelay = 50; // Debounce delay
↪ in milliseconds

// Variable to track the current color state
int colorState = 0; // 0: Off, 1: Red, 2: Green, 3: Blue

void setup() {
  // Set the RGB LED pins as OUTPUT
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // Set the button pin as INPUT_PULLUP
  pinMode(buttonPin, INPUT_PULLUP);

  // Initialize the RGB LED to be off
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);
}

void loop() {
  // Read the current state of the button
  int reading = digitalRead(buttonPin);

  // Check if the button state has changed (due to noise
  ↪ or pressing)
  if (reading != lastButtonState) {
    // Reset the debounce timer
    lastDebounceTime = millis();
  }

  // Check if the debounce delay has passed
  if ((millis() - lastDebounceTime) > debounceDelay) {
    // If the button state has changed
    if (reading != buttonState) {
      buttonState = reading;

      // If the button is pressed (LOW state)
      if (buttonState == LOW) {
        // Cycle through the color states
        colorState = (colorState + 1) % 4;

        // Update the RGB LED based on the current color
        ↪ state
        switch (colorState) {
          case 0: // Off
            digitalWrite(redPin, LOW);
            digitalWrite(greenPin, LOW);
            digitalWrite(bluePin, LOW);
            break;
          case 1: // Red
            digitalWrite(redPin, HIGH);
            digitalWrite(greenPin, LOW);
            digitalWrite(bluePin, LOW);
            break;
          case 2: // Green
            digitalWrite(redPin, LOW);
            digitalWrite(greenPin, HIGH);
            digitalWrite(bluePin, LOW);
            break;
          case 3: // Blue
            digitalWrite(redPin, LOW);
            digitalWrite(greenPin, LOW);
```

```
            digitalWrite(bluePin, HIGH);
            break;
        }
      }
    }
  }

  // Save the current reading as the last button state
  lastButtonState = reading;
}
```

**RGB LED**

```
// Define the pins for the RGB LED
const int redPin = 2;    // Red LED connected to pin 2
const int greenPin = 3;  // Green LED connected to pin 3
const int bluePin = 4;   // Blue LED connected to pin 4

void setup() {
  // Set the RGB LED pins as output
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // Initially turn off the RGB LED
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);
}

void loop() {
  // Red for 3 seconds
  digitalWrite(redPin, HIGH);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);
  delay(3000);

  // Yellow (Red + Green) for 3 seconds
  digitalWrite(redPin, HIGH);
  digitalWrite(greenPin, HIGH);
  digitalWrite(bluePin, LOW);
  delay(3000);

  // White (Red + Green + Blue) for 3 seconds
  digitalWrite(redPin, HIGH);
  digitalWrite(greenPin, HIGH);
  digitalWrite(bluePin, HIGH);
  delay(3000);

  // Turn off the LED for 3 seconds
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);
  delay(3000);
}
```

**Slide Potentiometer**

```
// Define the pins for the RGB LED
const int redPin = 2;    // Red LED connected to digital
↪  pin 2
const int greenPin = 3;  // Green LED connected to digital
↪  pin 3
const int bluePin = 4;   // Blue LED connected to digital
↪  pin 4

// Define the pin for the slide potentiometer
const int potPin = A3;   // Slide potentiometer connected
↪  to analog pin A3

void setup() {
  // Set the RGB LED pins as output
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // Initialize the RGB LED to be off
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);
}

void loop() {
  // Read the value from the slide potentiometer (0 to
  ↪  1023)
  int potValue = analogRead(potPin);

  // Determine the color based on the potentiometer value
  if (potValue < 341) {
    // First third of the range: Red
    digitalWrite(redPin, HIGH);
    digitalWrite(greenPin, LOW);
    digitalWrite(bluePin, LOW);
  } else if (potValue >= 341 && potValue < 682) {
    // Second third of the range: Green
    digitalWrite(redPin, LOW);
    digitalWrite(greenPin, HIGH);
    digitalWrite(bluePin, LOW);
  } else {
    // Final third of the range: Blue
    digitalWrite(redPin, LOW);
    digitalWrite(greenPin, LOW);
    digitalWrite(bluePin, HIGH);
  }
}
```

**7 Segment Display**

```
#include <Servo.h>

// Pin definitions
const int dataPin = A0;    // DS pin of 74HC595
const int latchPin = A1;   // STCP pin of 74HC595
const int clockPin = A2;   // SHCP pin of 74HC595

// 7-segment display digit patterns (common anode)
const byte digitPatterns[10] = {
  B11000000, // 0
  B11111001, // 1
```

```
  B10100100, // 2
  B10110000, // 3
  B10011001, // 4
  B10010010, // 5
  B10000010, // 6
  B11111000, // 7
  B10000000, // 8
  B10010000  // 9
};

unsigned long previousMillis = 0;  // Stores the last time the timer was updated
const long interval = 1000;       // Interval at which to update the timer (1 second)
int timer = 0;                     // Timer value

void setup() {
  // Set the pin modes
  pinMode(dataPin, OUTPUT);
  pinMode(latchPin, OUTPUT);
  pinMode(clockPin, OUTPUT);

  // Initialize the 7-segment display to be off
  digitalWrite(latchPin, LOW);
  shiftOut(dataPin, clockPin, MSBFIRST, B11111111); // Turn off all segments
  digitalWrite(latchPin, HIGH);
}

void loop() {
  unsigned long currentMillis = millis();

  // Check if 1 second has passed
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    // Increment the timer by 2 seconds
    timer += 2;

    // Determine what to display on the 7-segment display
    int displayValue;
    if (timer > 10) {
      displayValue = timer % 10; // Display only the last digit
    } else {
      displayValue = timer; // Display the full timer value
    }

    // Display the value on the 7-segment display
    digitalWrite(latchPin, LOW);
    shiftOut(dataPin, clockPin, MSBFIRST, digitPatterns[displayValue]);
    digitalWrite(latchPin, HIGH);
  }
}
```

**Servo**

```
#include <Servo.h>

// Define the pins
const int btn1Pin = 2;  // Button connected to digital pin 2
const int servo1Pin = 3; // Servo1 connected to digital pin 3
const int servo2Pin = 4; // Servo2 connected to digital pin 4

// Create servo objects
Servo servo1;
Servo servo2;

// Variables to store the servo positions
int servo1Pos = 0;
int servo2Pos = 180;

// Variables for button debouncing
int buttonState = HIGH;
int lastButtonState = HIGH;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50; // Debounce delay in milliseconds

void setup() {
  // Initialize the button pin as input with pull-up resistor
  pinMode(btn1Pin, INPUT_PULLUP);

  // Attach the servos to their respective pins
  servo1.attach(servo1Pin);
  servo2.attach(servo2Pin);

  // Set initial positions of the servos
  servo1.write(servo1Pos);
  servo2.write(servo2Pos);
}

void loop() {
  // Read the current state of the button
  int reading = digitalRead(btn1Pin);

  // Check if the button state has changed (due to noise or pressing)
  if (reading != lastButtonState) {
    // Reset the debouncing timer
    lastDebounceTime = millis();
  }

  // Check if the debounce delay has passed
  if ((millis() - lastDebounceTime) > debounceDelay) {
    // If the button state has changed
    if (reading != buttonState) {
      buttonState = reading;

      // Only toggle the servo positions if the button is pressed (LOW)
      if (buttonState == LOW) {
        // Toggle the servo positions
        if (servo1Pos == 0) {
```

```
          servo1Pos = 180;
          servo2Pos = 0;
        } else {
          servo1Pos = 0;
          servo2Pos = 180;
        }

        // Move the servos to their new positions
        servo1.write(servo1Pos);
        servo2.write(servo2Pos);
      }
    }
  }

  // Save the current reading as the last state for the
  ↪ next loop
  lastButtonState = reading;
}
```

**LED Bar Graph**

```
// Define the pins for bargraph1 and bargraph2
int bargraph1Pins[] = {2, 3, 4, 5, 6, 7, 8, 9, 10, 11};
int bargraph2Pins[] = {3, 4, 5, 6, 7, 8, 9, 10, 11, 12};

void setup() {
  // Initialize all LED pins as OUTPUT
  for (int i = 0; i < 10; i++) {
    pinMode(bargraph1Pins[i], OUTPUT);
    pinMode(bargraph2Pins[i], OUTPUT);
  }

  // Turn off all LEDs initially
  for (int i = 0; i < 10; i++) {
    digitalWrite(bargraph1Pins[i], LOW);
    digitalWrite(bargraph2Pins[i], LOW);
  }
}

void loop() {
  // Left to right sequence
  for (int i = 0; i < 10; i++) {
    // Turn on the current LED in bargraph1
    digitalWrite(bargraph1Pins[i], HIGH);

    // Turn on the previous LED in bargraph2 (if not the
    ↪ first LED)
    if (i > 0) {
      digitalWrite(bargraph2Pins[i - 1], HIGH);
    }

    // Turn off the LED in bargraph2 if bargraph1 is at the
    ↪ first LED
    if (i == 0) {
      digitalWrite(bargraph2Pins[9], LOW);
    }

    // Wait for 2 seconds
    delay(2000);

    // Turn off the current LED in bargraph1
    digitalWrite(bargraph1Pins[i], LOW);

    // Turn off the previous LED in bargraph2 (if not the
    ↪ first LED)
    if (i > 0) {
      digitalWrite(bargraph2Pins[i - 1], LOW);
    }
  }

  // Right to left sequence
  for (int i = 9; i >= 0; i--) {
    // Turn on the current LED in bargraph1
    digitalWrite(bargraph1Pins[i], HIGH);

    // Turn on the previous LED in bargraph2 (if not the
    ↪ first LED)
    if (i > 0) {
      digitalWrite(bargraph2Pins[i - 1], HIGH);
    }

    // Turn off the LED in bargraph2 if bargraph1 is at the
    ↪ first LED
    if (i == 0) {
      digitalWrite(bargraph2Pins[9], LOW);
    }

    // Wait for 2 seconds
    delay(2000);

    // Turn off the current LED in bargraph1
    digitalWrite(bargraph1Pins[i], LOW);

    // Turn off the previous LED in bargraph2 (if not the
    ↪ first LED)
    if (i > 0) {
      digitalWrite(bargraph2Pins[i - 1], LOW);
    }
  }
}
```

# I Prompt of R1-Compiler

**R1-Compiler Prompt**

The code is build failed because of the following error:

{ the error message from compiler }

Please fix the error
Note:
1. Your connections should in [CONNECTIONS] and [/CONNECTIONS]
2. Your code should in [ESP32 Code] and [/ESP32 Code]
3. Even if the origin connection or code is correct, you should still rewrite it.
4. First give the connection and then give the code.